

BIỆN PHÁP TINH GỌN REPOSITORY CHO AI CODING

Hướng dẫn giảm file dư thừa do AI sinh ra, giữ cấu trúc dự án rõ ràng và an toàn

Mục tiêu

Hạn chế file không cần thiết, code trùng lặp và tài liệu phát sinh sau mỗi prompt.

Phạm vi

Có thể áp dụng cho React, Node.js, TypeScript, Django, Laravel và dự án Machine Learning, với điều chỉnh theo từng stack.

Nguyên tắc

Ưu tiên sửa code hiện có; không xóa file nếu chưa xác minh mục đích và nơi sử dụng.

Tài liệu thực hành • Checklist + mẫu quy tắc + lệnh kiểm tra

1. Vấn đề cần giải quyết

Khi làm việc qua nhiều prompt, AI có thể tạo tài liệu phụ, bản sao file code, cấu hình trùng lặp hoặc file tạm. Những file này làm repository khó đọc, khiến AI dễ nhầm phiên bản và tăng chi phí bảo trì.

Mục tiêu không phải là giảm số lượng file bằng mọi giá, mà là giữ lại các file có vai trò rõ ràng và loại bỏ phần phát sinh không cần thiết một cách có kiểm chứng.

2. Nhận diện các nhóm file cần kiểm soát

Nhóm	Ví dụ	Biện pháp
Tài liệu AI sinh dư	IMPLEMENTATION_PLAN.md, FINAL_REPORT.md, SUMMARY.md, TASK_COMPLETED.md	Chỉ giữ tài liệu có giá trị lâu dài; không tạo nếu không được yêu cầu.
Code trùng lặp	userService_new.js, userService_v2.js, userService_fixed.js	Tìm và sửa module hiện có; không tạo bản sao thay thế tùy tiện.
File sinh tự động	__pycache__/, .pytest_cache/, log, cache, build output	Git ignore khi phù hợp; chỉ dọn file sinh tự động sau khi xác minh.
Quy tắc phân tán	README.md, AGENTS.md, AI_RULES.md chứa nội dung lặp	Duy trì một nguồn quy tắc chính; tránh sao chép nội dung giữa nhiều file.

3. Chuẩn hóa quy tắc cho AI

Đặt file AGENTS.md ở thư mục gốc nếu AI coding agent đang dùng hỗ trợ đọc file này. Với công cụ khác, hãy chuyển nội dung sang vị trí quy tắc mà công cụ đó hỗ trợ.

Mẫu AGENTS.md

```
# AI Coding Rules – Keep Repository Clean

## 1. Minimal Changes
- Inspect the existing codebase before making changes.
- Modify existing files whenever possible.
- Do not rewrite unrelated code.
- Preserve existing architecture, naming conventions, and coding style.
- Make the smallest change that correctly solves the requested task.

## 2. File Creation Policy
- Do not create a new file if the task can be solved by modifying an existing file.
- Search for an existing module with the same responsibility before creating a file.
- Create files only when necessary for functionality, testing, configuration, or maintainability.
- Never create duplicate files such as *_new, *_v2, *_fixed, or *_final.
- Do not create documentation, summaries, walkthroughs, or plans unless explicitly requested.
- Do not add configuration files without a demonstrated need.

## 3. Documentation
- Keep README.md as the main project overview.
- Update existing documentation only when necessary.
- Avoid duplicating information across README.md, AGENTS.md, and other documents.
- Create docs/ files only when they provide lasting value.

## 4. Code Reuse
- Reuse existing components, services, utilities, types, and helpers.
- Do not duplicate business logic.
- Avoid new dependencies when existing dependencies can solve the problem.
- Avoid unnecessary abstractions and overengineering.
```

5. Generated Files

- Do not manually create build output, cache files, temporary files, or logs unless required.
- Respect .gitignore.
- Do not delete files needed by deployment, tests, or runtime.
- Do not modify lockfiles unless dependency changes require it.

6. Safety

- Check a file's purpose before deleting, renaming, or overwriting it.
- Never remove user changes or unrelated code.
- Do not expose secrets or overwrite environment files.
- For potentially destructive cleanup, list candidates and explain the reason first.

7. Completion

- Run relevant tests or checks when possible.
- Report only: files created/modified/deleted, checks performed and results, remaining issues.
- Do not create a separate completion report file.

4. Cấu hình .gitignore

Mẫu dưới đây phù hợp làm điểm bắt đầu cho dự án Node.js/React/TypeScript. Hãy điều chỉnh theo yêu cầu build và deploy của repository thực tế.

```
# Dependencies
node_modules/

# Build output - ignore only if not required in Git
dist/
build/

# Environment variables and secrets
.env
.env.*
!.env.example

# Logs
*.log
npm-debug.log*
yarn-debug.log*
pnpm-debug.log*

# Cache
.cache/
.parcels-cache/
.eslintcache/
*.tsbuildinfo

# Test and coverage artifacts
coverage/
.nyc_output/

# OS
.DS_Store
Thumbs.db
```

- .gitignore không xóa file đã được Git theo dõi. Nếu file đã commit, cần xử lý riêng và cẩn trọng.
- Không ignore toàn bộ file .md, .json hoặc file cấu hình chung chỉ vì AI có thể tạo chúng; nhiều file trong số đó là thành phần thiết yếu.
- Với Django/Python, cần nhắc thêm __pycache__/, *.py[cod], .venv/ và .pytest_cache/ khi phù hợp.
- Chỉ ignore dist/ hoặc build/ nếu quy trình triển khai không yêu cầu lưu chúng trong Git.

5. Quy trình sau mỗi prompt AI

Bước 1 — Kiểm tra trước khi sửa

Yêu cầu AI đọc cấu trúc repository và tìm file đang phụ trách chức năng cần thay đổi.

Bước 2 — Chỉ sửa phần cần thiết

Ưu tiên sửa file hiện có. Chỉ tạo file mới khi có lý do kỹ thuật rõ ràng.

Bước 3 — Kiểm tra thay đổi

Xem danh sách file mới, file bị sửa và phạm vi diff trước khi chấp nhận kết quả.

Bước 4 — Xác minh trước khi dọn

Kiểm tra import, tham chiếu, build, test và deploy trước khi xóa file nghi vấn.

Bước 5 — Lưu mốc an toàn

Tạo commit hoặc bản sao lưu phù hợp trước khi thực hiện cleanup có tính phá hủy.

Lệnh kiểm tra nhanh

```
git status --short
git diff --stat
git diff --check
```

Các lệnh này giúp kiểm tra trạng thái file, quy mô thay đổi và lỗi khoảng trắng trong diff. Chúng không tự xác định file nào chắc chắn dư thừa.

6. Prompt dùng sau mỗi yêu cầu lập trình

Tuân thủ quy tắc tình huống repository:

- Luôn cấu trúc dự án và code hiện có trước khi chỉnh sửa.
- Ưu tiên sửa file hiện tại; không tạo file mới nếu không thực sự cần thiết.
- Không tạo README, báo cáo, summary, implementation plan, file backup hoặc file hướng dẫn bổ sung nếu tôi không yêu cầu.
- Không tạo file có hậu tố new, v2, fixed, final thay thế file cũ.
- Tái sử dụng component, service, utility và dependency có sẵn.
- Không thay đổi cấu trúc các phần không liên quan đến yêu cầu.
- Không xóa file chỉ vì tên có vẻ dư thừa; hãy kiểm tra nội dung sử dụng trước.
- Sau khi hoàn thành, liệt kê ngắn gọn các file đã tạo, sửa, xóa và kết quả kiểm tra.
- Nếu cần tạo file mới hoặc xóa file hiện có, hãy nêu lý do kỹ thuật rõ ràng.
- Chỉ thực hiện những thay đổi cần thiết để đáp ứng yêu cầu.

7. Quy trình rà soát repository hiện tại

- Liệt kê toàn bộ file mới và file chưa được commit.
- Phân loại file thành: cần giữ, file sinh tự động, file trùng lặp, chưa xác định.
- Tìm nơi import, tham chiếu hoặc sử dụng từng file nghi vấn.
- Kiểm tra build, test và deploy trước khi xóa.
- Tạo commit hoặc bản sao lưu an toàn.
- Xóa từng nhóm nhỏ, sau đó chạy lại kiểm thử và xem git diff.

Không xóa hàng loạt chỉ dựa trên tên file. File có vẻ thừa vẫn có thể được dùng bởi runtime, script, CI/CD hoặc quy trình deploy.

8. Checklist áp dụng

- Đã có file quy tắc AI ở vị trí phù hợp với công cụ đang dùng.
- AI đã kiểm tra cấu trúc dự án trước khi tạo file mới.
- Không có file *_new, *_v2, *_fixed hoặc *_final không cần thiết.
- Tài liệu không bị lặp nội dung giữa nhiều file.
- .gitignore phù hợp với stack và quy trình deploy.
- Đã xem git status --short và git diff --stat.
- File dự định xóa đã được kiểm tra nơi sử dụng.
- Đã chạy các kiểm tra phù hợp sau khi dọn.

9. Nguyên tắc cốt lõi

Kết hợp bốn lớp: (1) quy tắc AI, (2) .gitignore, (3) kiểm tra diff sau mỗi prompt, và (4) cleanup có xác minh. Mỗi lớp giải quyết một vấn đề khác nhau; không lớp nào thay thế hoàn toàn các lớp còn lại.

Áp dụng linh hoạt theo từng repository. Mục tiêu cuối cùng là giữ nguyên chức năng, giảm phần phát sinh không cần thiết và giúp cả lập trình viên lẫn AI hiểu đúng cấu trúc dự án.